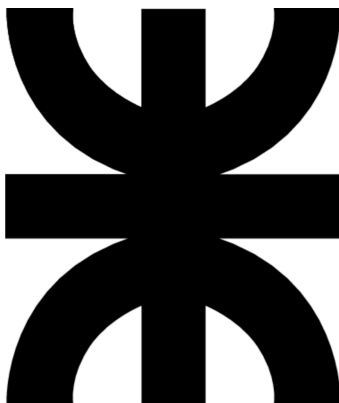


Proyecto Smart Home



Integrantes:

- Fernandez, Orianna Del Rosario
- Aratto, José Alejandro
- Cerqueiro, Carabajal Octavio
- Taus, Fernando Andrés
- Hochmuth, Nehueln Lautaro
- Goy, Bruno Sebastian

Asignatura: Sintaxis y Semántica de los Lenguajes

Carrera: Ing. en Sistemas de Información

Primer cuatrimestre

Curso académico: 2026

Universidad y regional: UTN FRRe

Lugar y fecha: Resistencia, 2026

Índice

1. Introducción.....	3
1.1. Descripción de lo implementado.....	3
1.2. Información y Requerimientos de Software.....	4
2. Conformación de Grupo.....	4
2.1. Integrantes.....	4
2.2. Matriz de Habilidades.....	5
2.3. Identidad.....	5
2.4. Alianza.....	6
3. Gramática.....	6
3.1. Símbolos de la Gramática.....	6
3.1.1. Símbolos Terminales.....	6
3.1.2. Símbolos No Terminales.....	7
3.2. Reglas de Producción.....	8
3.2.1 Regla inicial.....	8
3.2.2 Lista de sentencias.....	8
3.2.3 Tipos de sentencia.....	8
3.2.4 Estructura del bloque WHEN.....	9
3.2.5 Estructura del bloque EVERY.....	9
3.2.6 Estructura del bloque IF.....	9
3.2.7 Reglas de asignación.....	9
3.2.8 Estructura de bloque.....	10
3.2.9 Regla general de condición.....	10
3.2.10 Expresiones lógicas OR.....	11
3.2.11 Expresiones lógicas AND.....	11
3.2.12 Expresiones lógicas NOT.....	11
3.2.13 Condiciones primarias.....	11
3.2.14 Reglas de comparación.....	11
3.2.15 Expresiones temporales.....	13
3.2.16 Operadores.....	13
3.3. Representación Gráfica de Árbol Sintáctico (AST).....	13
4. Analisis Lexico.....	14
4.1 Estructura y Funcionamiento de Lexer.l.....	14
4.1.1. Configuración y Tolerancia (Directivas de Flex).....	15
4.1.2. Definiciones Regulares.....	15
4.1.3. Reconocimiento de Tokens y Lexemas.....	15
4.1.3. Rastreador de Errores (YY_USER_ACTION).....	15
5. Analisis Sintactico.....	15
5.1. Estructura y Funcionamiento del Parser.y.....	16
5.1.1. Declaraciones y Puentes:.....	16
5.1.2. Definición de Tokens.....	16
5.1.3. Definición de Reglas de Producción.....	16
5.1.4. Traducción Dirigida por la Semántica.....	17
5.1.5. Manejo de Errores Sintácticos.....	17

6. Traducción a HTML.....	17
6.1 Implementación.....	17
7. Funciones Auxiliares.....	18
7.1. Módulo Principal main.c.....	18
7.2. Analizador lexico Lexer.l.....	18
7.3. Analizador Sintáctico Parser.y.....	19
8. Modo de Obtención y Estructura de Proyecto.....	19
8.1. Proceso de Construcción.....	19
8.2 Explicación de Archivos fuentes.....	19
9. Modos de Ejecución del Intérprete.....	20
9.1 Modo Interactivo.....	20
9.2 Modo Lectura de Archivo.....	20
10. Conclusiones.....	21
11. Bibliografía.....	21
11.1. Documentación Obligatoria de la Cátedra.....	21
11.2. Software.....	21
11.3. Recursos Audiovisuales (Tutoriales).....	21
11.4. Asistencia por Inteligencia Artificial (IA).....	22
12. Anexos.....	22

1. Introducción

Este trabajo práctico integrador detalla el desarrollo de un compilador/intérprete para un lenguaje de dominio específico (DSL) orientado a la automatización de hogares inteligentes (Smart Home). El objetivo principal de la herramienta es procesar scripts escritos en este lenguaje personalizado (con extensión .smart) y traducirlos dinámicamente a un panel de control visual en formato HTML.

Este documento se divide en las siguientes secciones principales:

- **Análisis Léxico:** Descripción de la tokenización y reconocimiento de patrones.
- **Análisis Sintáctico y Semántico:** Definición de la gramática libre de contexto y las reglas de validación.
- **Generación de Código:** Explicación del esquema de traducción dirigida por la sintaxis para la construcción del documento HTML.
- **Manejo de Errores:** Estrategias implementadas para la detección y recuperación de errores léxicos y sintácticos.
- **Manual de Uso:** Instrucciones para la compilación y ejecución del proyecto.

1.1. Descripción de lo implementado

La solución fue implementada utilizando un enfoque clásico de compilación en dos etapas principales, apoyado por herramientas de generación automática de analizadores y manejo mediante un programa principal en lenguaje C.

- **Analizador Léxico (Flex - Lexer.l): El Lector**
Es el encargado de leer letra por letra el archivo que escribimos y agruparlas en "palabras" que el sistema pueda entender, lo que llamamos tokens. Le enseñamos a reconocer nuestras palabras clave, los nombres de los aparatos como sensores, luces, alarmas, etc. y datos más complejos como horas, fechas o temperaturas. Además, le agregamos un rastreador que anota en qué línea estamos; así, si nos equivocamos al escribir, el programa nos puede decir exactamente en qué renglón está el problema.
- **Analizador Sintáctico y Traductor (Bison - Parser.y): El Traductor y Corrector**
Esta parte se encarga de revisar que las palabras que encontró el Lexer formen "oraciones" que tengan sentido. Revisa que estemos armando bien las reglas, por ejemplo, que a un WHEN le siga una condición y luego un DO. En esta parte se hace un trabajo doble: a medida que lee y confirma que una oración está bien escrita, automáticamente va escribiendo el diseño de la página web (HTML y CSS) para mostrarlo en el panel de control. También le pusimos un "salvavidas": si encuentra un error de tipeo o de sintaxis, no tira todo por la borda ni se detiene; anota el error, se recupera y sigue revisando el resto del archivo.
- **Controlador Principal (C - main.c):**
Es el programa principal que coordina todo. Primero, hace de guardia de seguridad y verifica que el archivo que le pasamos termine en .smart. Si todo está en orden, crea el archivo web vacío, le pone los estilos visuales básicos y le da la orden al Analizador Sintáctico de que empiece a traducir. Cuando termina todo el proceso, nos da un reporte final: si todo salió perfecto, nos entrega nuestra página web lista; pero si encuentra errores en el camino, destruye automáticamente el archivo a medio hacer para no entregarnos un código roto.

1.2. Información y Requerimientos de Software

Para compilar y ejecutar el intérprete de manera correcta, es necesario contar con el siguiente entorno y conjunto de herramientas:

- **Lenguaje Base:** C (Estándar C99 o superior).
- **Generador de Analizador Léxico:** Flex (Fast Lexical Analyzer Generator) - [versión 2.6.4].
- **Generador de Analizador Sintáctico:** Bison (GNU Parser Generator) - [3.8.2].
- **Compilador de C:** GCC (GNU Compiler Collection). En entornos Windows, se recomienda utilizar MinGW o MSYS2.
- **Plataforma de Ejecución:** Multiplataforma (Windows / Linux). El código incluye directivas de preprocesador específicas (`#ifdef _WIN32`) para garantizar la correcta visualización de mensajes y manejo de la consola en sistemas operativos Windows mediante la API `windows.h`.
- **Editor de Código / IDE:** Visual Studio Code.

Proceso de Compilación:

1. Generar el código en C del lexer: **flex Lexer.l**
2. Generar el código en C del parser: **bison -d Parser.y**
3. Compilar el proyecto completo: **gcc -o Interprete Parser.tab.c lex.yy.c main.c**

Ejecución:

El programa se ejecuta por línea de comandos pasando como único argumento el archivo fuente a analizar (ejemplo estando en la carpeta principal): **./bin/Interprete.exe ./prueba/ejemplo.smart**

2. Conformación de Grupo

2.1. Integrantes

- Fernandez, Orianna Del Rosario
- Aratto, José Alejandro
- Cerqueiro, Carabajal Octavio
- Taus, Fernando Andrés
- Hochmuth, Nehueln Lautaro
- Goy, Bruno Sebastian

2.2. Matriz de Habilidades

Integrantes	Inglés	Edición de video	Habilidades de Programación	Búsqueda de información	Documentación	Idear soluciones
Aratto José Alejandro	O	★	★	O		★
Taus Fernando Andrés	★	★	★	O	O	★
Hochmuth Nehueln Lautaro	★	O	★	★		O
Goy Bruno Sebastian	★		O	★	O	★
Cerqueiro Carabajal Octavio	★	O	O	★	★	★
Fernandez Orianna Del Rosario	★	★	★		O	O

2.3. Identidad

Nombre: C.mánticos

Logo:



2.4. Alianza

Como grupo, acordamos organizarnos de manera colaborativa y responsable para el desarrollo del trabajo práctico. Nuestro objetivo principal es lograr un proyecto sólido, bien documentado y funcional, distribuyendo las tareas de forma equitativa y aprovechando las fortalezas individuales de cada integrante. Nos proponemos que al menos dos personas del equipo puedan desenvolverse con fluidez en tareas claves como la programación, documentación y planificación del proyecto.

Nos comprometemos a mantener reuniones de trabajo y seguimiento al menos tres veces por semana, priorizando los encuentros nocturnos durante la semana y en la mañana los fines de semana. Estas reuniones se realizan de manera virtual a través de Discord, y utilizaremos GitHub como plataforma principal para el seguimiento del proyecto y mantenerlo siempre actualizado.

Nos comunicaremos de forma constante y fluida por Discord, y en caso de ser necesario, también por otros canales complementarios como WhatsApp.

Todos los integrantes del equipo nos comprometemos a trabajar de forma constante a lo largo del plazo establecido, evitando dejar tareas para último momento, con el objetivo de mantener una buena calidad del trabajo y reducir el estrés asociado a la entrega.

Las metas y valores comunes en nuestro equipo son:

- Responsabilidad individual y grupal.
- Buena comunicación continua.
- Organización y planificación para cumplir en tiempo y forma las entregas.
- Apoyo en dificultades generales.
- Abierto a escuchar opiniones distintas a lo largo del desarrollo y debatir soluciones.

3. Gramática

Para el análisis sintáctico del intérprete Smart-Home, se diseñó una Gramática Libre de Contexto (GLC). Esta gramática define formalmente la estructura y las reglas sintácticas que un script con extensión .smart debe seguir para ser considerado válido.

3.1. Símbolos de la Gramática

3.1.1. Símbolos Terminales

Son los tokens indivisibles identificados por el Analizador Léxico (Lexer) y enviados al Analizador Sintáctico. En nuestra gramática, los representamos convencionalmente en letras mayúsculas. A continuación, se detallan todos los tokens definidos en el sistema agrupados por su funcionalidad:

1. **Palabras reservadas:** Son las sentencias que definen el flujo y comportamiento reactivo del script.
TK_WHEN, TK_IF, TK_THEN, TK_ELSE, TK_DO, TK_END, TK EVERY
2. **Operadores lógicos:** Utilizados para anidar y combinar múltiples condiciones.
TK_AND, TK_OR, TK_NOT

3. Identificadores de Sensores (Entradas):

SENSOR_TEMPERATURA_ID, SENSOR_HUMEDAD_ID, SENSOR_LUZ_ID,
SENSOR_MOVIMIENTO_ID, SENSOR_HUMO_ID, RELOJ_ID

4. Identificadores de Actuadores (Salidas):

FOCO_ID, AIRE_ID, PERSIANA_ID, CERRADURA_ID, ALTAVOZ_ID, ALARMA_ID

5. Atributos de Dispositivos: Definen las propiedades que se pueden leer de los sensores o modificar en los actuadores.

TK_ESTADO, TK_ACTIVADA, TK_BRILLO, TK_COLOR, TK_MODO,
TK_TEMP_OBJ, TK_TEMP_OBJETIVO, TK_TEMP_ACT, TK_POSICION,
TK_VOLUMEN, TK_MUTE, TK_MENSAJE, TK_EMAIL_NOTIF, TK_EMAIL,
TK_HORA, TK_FECHA

6. Operadores y Puntuación: Símbolos matemáticos y estructurales del lenguaje.

- OP_PUNTO (.)
- OP_IGUALDAD (== , !=)
- OP_RELACIONAL (< , > , <= , >=)
- ASIGNACION (=)

7. Literales y Unidades: Valores de datos crudos extraídos por el Lexer y pasados como cadenas (strings) para su uso semántico.

- BOOLEANO (TRUE / FALSE / ON / OFF)
- PORCENTAJE (ej: 80%)
- TEMPERATURA (ej: 24°C)
- ILUMINANCIA (ej: 250lux)
- TIEMPO (ej: 30m)
- VALOR_HORA (ej: 22:00)
- VALOR_FECHA (ej: 21/06/2026)
- EMAIL (ej: usuario@smarthome.com)
- TEXTO (Cadenas entre comillas, ej: "Son las 22hs")

3.1.2. Símbolos No Terminales

Son las variables estructurales o abstracciones que se derivan en otras estructuras o tokens. Representan los conceptos del lenguaje y se escriben en minúsculas.

1. Estructura Base:

- programa, lista_sentencias, sentencia, bloque

2. Condicionales y Eventos

- when, every, if_sentencia

3. Expresiones Lógicas (Condiciones):

- condicion, expresion_or, expresion_and, expresion_not, primaria_condicion

4. Operaciones de Comparación (Lectura de Sensores y Estados):

- comparacion
- comparacion_temp
- comparacion_hum

- comparacion_luz
- comparacion_mov
- comparacion_humo
- comparacion_reloj
- comparacion_actuador

5. Operaciones de Asignación (Control de Actuadores):

- asignacion
- asignacion_foco
- asignacion_aire
- asignacion_persiana
- asignacion_cerradura
- asignacion_altavoz
- asignacion_alarma

6. Atributos Adicionales:

- tiempo
- operador_comp

3.2. Reglas de Producción

A continuación se detallan las reglas de derivación de nuestra gramática, utilizando notación. El axioma principal o símbolo inicial de nuestra gramática es programa.

3.2.1 Regla inicial

programa → lista_sentencias

Descripción: Es el punto de entrada principal del compilador. Define que un archivo .smart válido y completo está compuesto fundamentalmente por un conjunto de instrucciones o sentencias.

3.2.2 Lista de sentencias

lista_sentencias → sentencia lista_sentencias

lista_sentencias → sentencia

Descripción: Define la estructura recursiva del código. Permite que el programa procese desde una única instrucción hasta múltiples sentencias concatenadas (una debajo de la otra) sin límite de cantidad.

3.2.3 Tipos de sentencia

sentencia → when

sentencia → every

sentencia → if

sentencia → asignacion

Descripción: Actúa como un distribuidor o clasificador principal. Establece que cualquier instrucción individual en el script debe pertenecer a una de las cuatro categorías válidas: un evento reactivo (**when**), un ciclo temporal (**every**), un condicional clásico (**if**) o una acción directa de cambio de estado (**asignacion**).

3.2.4 Estructura del bloque WHEN

when → WHEN condicion DO bloque END

Descripción: Define el corazón reactivo del sistema domótico. Espera la palabra reservada **WHEN**, evalúa una condición lógica y, en caso de cumplirse, ejecuta las acciones contenidas en el bloque, cerrando la estructura con END.

3.2.5 Estructura del bloque EVERY

every → EVERY tiempo DO bloque END

Descripción: Define una estructura de repetición o cronograma. Permite ejecutar un grupo de acciones (**bloque**) de manera periódica, dictado por el intervalo o momento definido en la variable tiempo.

3.2.6 Estructura del bloque IF

if → IF condicion THEN bloque ELSE bloque END

if → IF condicion THEN bloque END

Descripción: Define las ramificaciones lógicas clásicas. Permite ejecutar un bloque de acciones si la condición es verdadera y provee la capacidad (**opcional**) de ejecutar un bloque alternativo (**ELSE**) en caso de ser falsa.

3.2.7 Reglas de asignación

asignacion → asignacion_foco

asignacion → asignacion_aire

asignacion → asignacion_persiana

asignacion → asignacion_cerradura

asignacion → asignacion_altavoz

asignacion → asignacion_alarma

asignacion_foco → FOCO_ID . color = VALOR_COLOR

VALOR_COLOR → blanco | rojo | azul

asignacion_foco → FOCO_ID . estado = BOOLEANO

asignacion_foco → FOCO_ID . brillo = PORCENTAJE

asignacion_aire → AIRE_ID . temp_obj = TEMPERATURA

asignacion_aire → AIRE_ID . temp_objetivo = TEMPERATURA

asignacion_aire → AIRE_ID . estado = BOOLEANO

asignacion_aire → AIRE_ID . modo = modo_aire

modo_aire → FRIO | CALOR | VENT

asignacion_persiana → PERSIANA_ID . posicion = PORCENTAJE

asignacion_cerradura → CERRADURA_ID . estado = BOOLEANO

asignacion_altavoz → ALTAVOZ_ID . volumen = PORCENTAJE

asignacion_altavoz → ALTAVOZ_ID . mute = BOOLEANO

asignacion_altavoz → ALTAVOZ_ID . mensaje = TEXTO

asignacion_altavoz → ALTAVOZ_ID . email_notif = EMAIL

asignacion_altavoz → ALTAVOZ_ID . email = EMAIL

asignacion_alarma → ALARMA_ID . estado = BOOLEANO

asignacion_alarma → ALARMA_ID . activada = BOOLEANO

Descripción: Determina cómo se modifican los atributos físicos de los actuadores de la casa. Utilizando una sintaxis estructurada con notación de punto, esta regla válida que sólo se asignen valores lógicos a los dispositivos correctos (por ejemplo, encender un foco, cambiar la posición de una persiana a un porcentaje o enviar un email mediante un altavoz).

3.2.8 Estructura de bloque

bloque → sentencia bloque

bloque → sentencia

Descripción: Similar a lista_sentencias, pero de alcance local. Permite agrupar una o más instrucciones dentro de los "cuerpos" de las estructuras de control (WHEN, EVERY, IF) de manera recursiva.

3.2.9 Regla general de condición

condicion → expresion_or

Descripción: Es el punto de partida para evaluar las reglas lógicas del lenguaje. Delega inmediatamente la evaluación al operador lógico de menor jerarquía (**OR**) para respetar la precedencia correcta.

3.2.10 Expresiones lógicas OR

expresion_or → expresion_and OR expresion_or

expresion_or → expresion_and

Descripción: Maneja la disyunción lógica. Si hay varias condiciones, permite que el bloque se ejecute si al menos una de las ramas es verdadera.

3.2.11 Expresiones lógicas AND

expresion_and → expresion_not AND expresion_and

expresion_and → expresion_not

Descripción: Maneja la conjunción lógica. Tiene mayor prioridad de evaluación matemática que el operador **OR**. Exige que ambas partes de la expresión se cumplan para devolver verdadero.

3.2.12 Expresiones lógicas NOT

expresion_not → NOT expresion_not

expresion_not → primaria_condicion

Descripción: Maneja la negación unitaria. Posee la mayor prioridad entre los operadores lógicos, revirtiendo el valor de verdad de la expresión primaria que le sigue de forma inmediata.

3.2.13 Condiciones primarias

primaria_condicion → comparacion

primaria_condicion → BOOLEANO

Descripción: Representa el nivel más atómico de una expresión lógica. Se resuelve directamente en una comparación física entre dispositivos, o directamente en un literal booleano predefinido (**TRUE / FALSE**).

3.2.14 Reglas de comparación

comparacion → comparacion_temp

comparacion → comparacion_hum

comparacion → comparacion_luz

comparacion → comparacion_mov

comparacion → comparacion_humo

comparacion → comparacion_reloj

comparacion → comparacion_actuador

comparacion_temp → SENSOR_TEMPERATURA_ID OPERADOR_COMP TEMPERATURA

comparacion_hum → SENSOR_HUMEDAD_ID OPERADOR_COMP PORCENTAJE

comparacion_luz → SENSOR_LUZ_ID OPERADOR_COMP ILUMINANCIA

comparacion_mov → SENSOR_MOVIMIENTO_ID OP_IGUALDAD BOOLEANO

comparacion_humo → SENSOR_HUMO_ID OP_IGUALDAD BOOLEANO

comparacion_reloj → RELOJ_ID . hora OPERADOR_COMP VALOR_HORA

comparacion_reloj → RELOJ_ID . fecha OPERADOR_COMP VALOR_FECHA

comparacion_actuador → FOCO_ID . estado OP_IGUALDAD BOOLEANO

comparacion_actuador → FOCO_ID . brillo OPERADOR_COMP PORCENTAJE

comparacion_actuador → FOCO_ID . color OP_IGUALDAD VALOR_COLOR

comparacion_actuador → AIRE_ID . estado OP_IGUALDAD BOOLEANO

comparacion_actuador → AIRE_ID . modo OP_IGUALDAD modo_aire

comparacion_actuador → AIRE_ID . temp_obj OPERADOR_COMP TEMPERATURA

comparacion_actuador → AIRE_ID . temp_act OPERADOR_COMP TEMPERATURA

comparacion_actuador → PERSIANA_ID . posicion OPERADOR_COMP PORCENTAJE

comparacion_actuador → CERRADURA_ID . estado OP_IGUALDAD BOOLEANO

comparacion_actuador → ALTAVOZ_ID . volumen OPERADOR_COMP PORCENTAJE

comparacion_actuador → ALTAVOZ_ID . mute OP_IGUALDAD BOOLEANO

comparacion_actuador → ALARMA_ID . estado OP_IGUALDAD BOOLEANO

comparacion_actuador → ALARMA_ID . activada OP_IGUALDAD BOOLEANO

Descripción: Define las lecturas de contexto y estados. Valida sintácticamente las comparaciones entre las métricas ambientales reales captadas por los sensores (temperatura, luz, presencia de humo, hora del reloj) o los estados actuales de los actuadores y los valores definidos en el script, utilizando operadores relacionales (ej. SENSOR_LUZ_ID < 250lux).

3.2.15 Expresiones temporales

tiempo → TIEMPO

tiempo → VALOR_HORA

Descripción: Especifica los formatos de magnitud cronológica aceptados por el intérprete, permitiendo diferenciar entre duraciones relativas (lapsos de minutos u horas) y tiempos absolutos del reloj.

3.2.16 Operadores

operador_comp → OP_IGUALDAD | OP_RELACIONAL

Descripción: Símbolo no terminal auxiliar que unifica los componentes léxicos de equivalencia (**OP_IGUALDAD**) y de magnitud (**OP_RELACIONAL**). Su objetivo es simplificar la sintaxis de la gramática, permitiendo que las reglas de lectura de los sensores y estados de los actuadores acepten cualquier operador lógico-matemático bajo una misma abstracción.

Nota Técnica sobre el alcance Sintáctico: Cabe destacar que la Gramática Libre de Contexto (GLC) aquí definida no incluye reglas de producción para la formación de los símbolos terminales. Esta responsabilidad fue delegada en su totalidad al Analizador Léxico (mediante expresiones regulares), el cual entrega al Analizador Sintáctico los tokens ya conformados como unidades atómicas indivisibles.

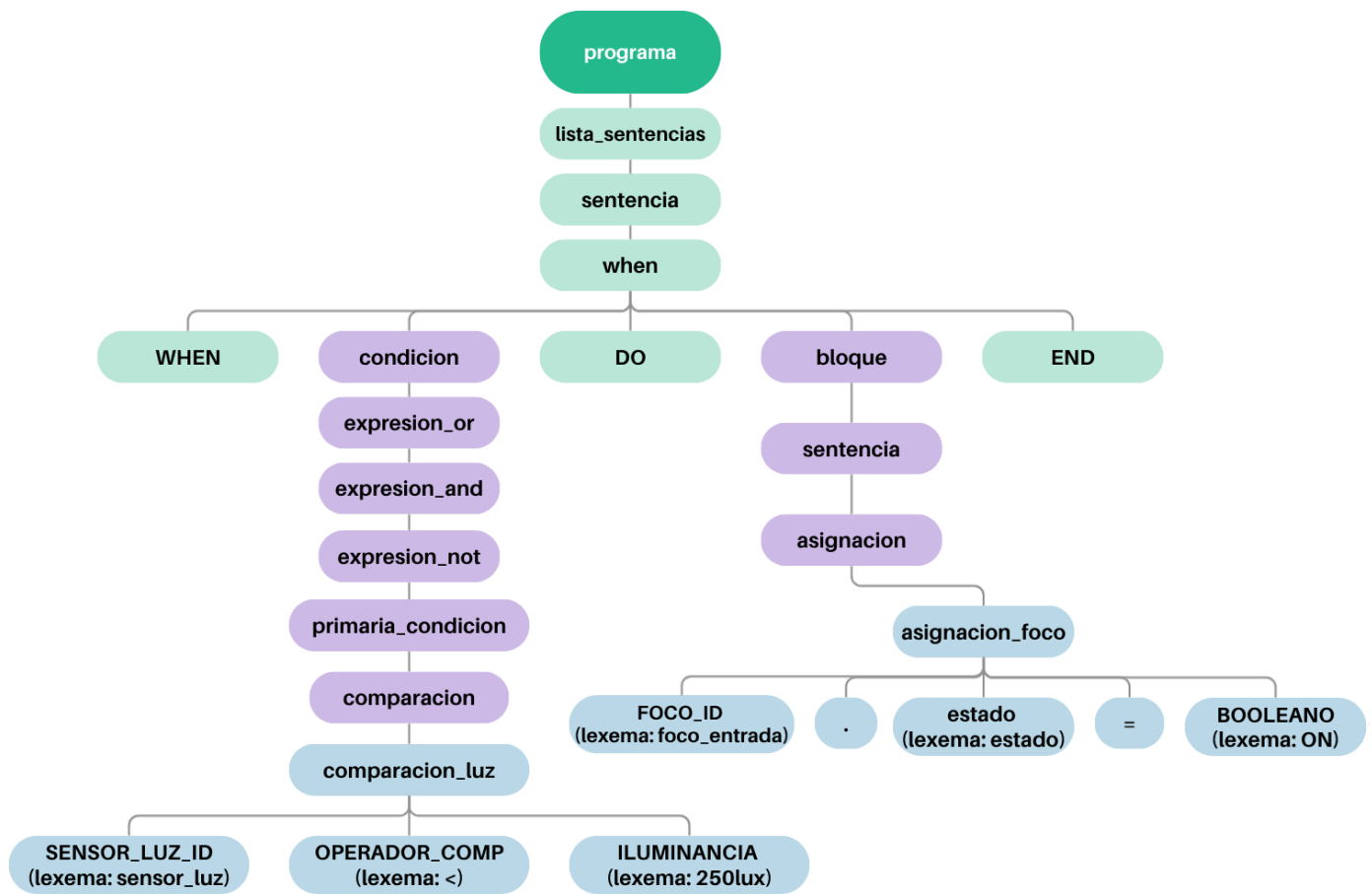
3.3. Representación Gráfica de Árbol Sintáctico (AST)

Un Árbol de Derivación (o Árbol de Análisis Sintáctico) es la representación visual de cómo el Parser agrupa los tokens secuenciales recibidos del Lexer aplicando las reglas de la Gramática Libre de Contexto. En esta estructura, la raíz del árbol es siempre nuestro axioma inicial (programa), las ramas intermedias son los Símbolos No Terminales (las abstracciones lógicas), y las hojas finales son los Símbolos Terminales (los tokens indivisibles).

Para ilustrar el funcionamiento lógico de nuestro intérprete, tomamos como caso de estudio la siguiente sentencia válida del dominio domótico:

WHEN sensor_luz < 250lux DO foco_entrada.estado = ON END

A continuación, se presenta el esquema del árbol sintáctico generado al procesar dicha instrucción, evidenciando la derivación en cascada para la precedencia lógica y la correcta agrupación de los bloques de asignación:



4. Analisis Lexico

El analizador léxico, implementado en el archivo **Lexer.I** utilizando la herramienta Flex, actúa como el "lector" de nuestro intérprete. Su trabajo principal es leer el código fuente escrito por el usuario letra por letra, agrupar esos caracteres en bloques lógicos con significado, llamados tokens, y enviarlos al Analizador Sintáctico.

¿Qué es Flex?

Flex (Fast Lexical Analyzer Generator) es una herramienta de software diseñada para generar analizadores léxicos rápidos y eficientes. Es un programa que escribe otro programa, en nuestro caso código fuente en lenguaje C, capaz de procesar e identificar patrones en un texto.

Flex nos permite trabajar a un nivel más abstracto y declarativo mediante el uso de **Expresiones Regulares**.

4.1 Estructura y Funcionamiento de Lexer.I

Para lograr que nuestro analizador léxico reconozca todos los componentes del lenguaje doméstico, estructuramos el archivo en cuatro partes fundamentales:

4.1.1. Configuración y Tolerancia (Directivas de Flex)

Al inicio del archivo, establecimos algunas reglas base para que el intérprete sea amigable con el usuario.

- Utilizamos **%option case-insensitive** para que el lenguaje no distinga entre mayúsculas y minúsculas (permitiendo que el usuario escriba WHEN, when o wHeN y el sistema lo entienda igual).
- Activamos **%option 8bit** para soportar caracteres especiales, algo indispensable en nuestro dominio para poder leer correctamente el símbolo de grados en las temperaturas (ej. 24°C).

4.1.2. Definiciones Regulares

Le indicamos al Lexer a reconocer estructuras usando Expresiones Regulares. De esta forma, el sistema atrapa automáticamente formatos complejos como:

- **Magnitudes:** Temperaturas (ej. -5°C), porcentajes (80%) e iluminancia (250lux).
- **Tiempos:** Horas de reloj (22:00), fechas exactas (21/06/2026) y lapsos de tiempo cortos (30m).
- **Datos de usuario:** Direcciones de correo electrónico (usuario@smarthome.com).

4.1.3. Reconocimiento de Tokens y Lexemas

Esta es la parte central del archivo, donde emparejamos el texto leído con las "piezas de Lego" que el Analizador Sintáctico está esperando:

- **Palabras reservadas:** Al leer términos de control estructural (WHEN, DO, EVERY), el Lexer simplemente envía una señal de confirmación (ej. devuelve TK_WHEN).
- **Captura de valores (yyval):** Cuando el Lexer encuentra un identificador de un dispositivo (ej. sensor_luz) o un valor útil (ej. un color o una temperatura), no solo avisa qué tipo de token encontró, sino que utiliza la función `strdup(yytext)` para hacer una "fotocopia" del texto exacto que escribió el usuario. Esta fotocopia viaja hacia el Parser y es vital para poder generar el diseño HTML final con los datos correctos.

4.1.3. Rastreador de Errores (YY_USER_ACTION)

Para que nuestro intérprete no colapse ciegamente ante un error de tipeo, implementamos una macro especial que actúa como una grabadora en segundo plano. Cada vez que el Lexer avanza y lee un token válido, guarda ese fragmento de texto en un buffer (una memoria temporal) que representa la línea actual. Si el programa encuentra un error más adelante, utiliza esta información almacenada para imprimirle al usuario exactamente en qué línea y lugar del texto se equivocó.

5. Analisis Sintactico

El analizador sintáctico, desarrollado en el archivo **Parser.y** mediante la herramienta **GNU Bison**, es el "cerebro gramatical" de nuestro proyecto. Su función primordial es recibir el

flujo de tokens que le envía Flex y verificar que estén ordenados siguiendo las reglas estrictas de la jerarquía que definimos en nuestra Gramática Libre de Contexto (GLC).

¿Qué es Bison?

Bison es un generador de analizadores sintácticos (parsers) de propósito general. Mientras que Flex se encarga de agrupar letras en palabras (tokens), Bison se encarga de analizar cómo esas palabras se organizan para formar estructuras con sentido.

Bison toma la Gramática Libre de Contexto (GLC) que definimos y genera automáticamente el código en C que implementa el analizador. Este analizador es capaz de verificar si un archivo fuente sigue las reglas gramaticales del lenguaje y permite ejecutar acciones cuando reconoce una estructura específica.

5.1. Estructura y Funcionamiento del Parser.y

Para lograr que nuestro analizador sintáctico reconozca patrones y verificar que estén bien, estructuramos el archivo Parser.y en 4 partes fundamentales:

5.1.1. Declaraciones y Puentes:

En la cabecera del archivo, incluimos las librerías estándares de C necesarias y realizamos el enlace con las variables globales y externas compartidas con el resto del software. Entre ellas, el contador de líneas (**linea**), el acumulador de errores (**cant_errores**), el búfer de texto (**linea_actual**) y el puntero al archivo web de salida (**f_html**). Asimismo, se define la estructura **%union** para establecer que los valores semánticos de los tokens contengan cadenas de caracteres (**char***), permitiendo que los nombres de los sensores y los valores viajen intactos.

5.1.2. Definición de Tokens

En esta sección le indicamos a Bison cuáles son las palabras del vocabulario que va a recibir. Declaramos tanto los identificadores de dispositivos (**SENSOR_LUZ_ID**, **FOCO_ID**, **etc.**), como las palabras clave de control estructural (**TK_WHEN**, **TK_DO**, **TK_IF**, **TK_EVERY**) y los literales de medición (**TEMPERATURA**, **ILUMINANCIA**, **PORCENTAJE**), asegurando un acoplamiento perfecto con los retornos generados por Flex.

5.1.3. Definición de Reglas de Producción

Es el cuerpo principal del archivo. Aquí se describen las combinaciones válidas del lenguaje. Como el Parser opera mediante un algoritmo de **análisis sintáctico ascendente**, las reglas se van reduciendo secuencialmente desde las estructuras más pequeñas hasta alcanzar el axioma inicial (**programa**). Un aspecto clave de este diseño fue estructurar las expresiones lógicas en forma de cascada recursiva (**condicion** → **expresion_or** → **expresion_and** → **expresion_not** → **primaria_condicion**). Esta estrategia teórica es fundamental porque resuelve de manera automática la precedencia de operadores lógicos (el **NOT** evalúa antes que el **AND**, y este antes que el **OR**), eliminando cualquier tipo de ambigüedad al momento de construir el árbol sintáctico.

5.1.4. Traducción Dirigida por la Semántica

Nuestro parser no se limita únicamente a validar si el código es correcto; funciona también como un traductor activo. Al final de cada regla de producción, colocamos bloques de código en C nativo encerrados entre llaves “{}”. Cada vez que Bison reconoce con éxito que una regla se cumplió (por ejemplo, al validar que la intensidad de una luz cambió de valor), ejecuta esa acción y automáticamente genera bloques de código HTML y hojas de estilo CSS en línea directamente en el archivo físico de salida. De esta forma, el proceso de análisis gramatical y la construcción visual del panel web ocurren en simultáneo en una sola pasada.

5.1.5. Manejo de Errores Sintácticos

Para evitar que el intérprete se detenga ante el primer descuido tipográfico del usuario, insertamos el token especial error en puntos críticos de la gramática. Esto se complementa con las funciones de recuperación nativas de Bison `yyperror` y `yyclearin`. Gracias a este mecanismo, si falta un punto o se tipea mal un atributo, el sistema emite el aviso correspondiente, descarta la instrucción defectuosa para no corromper el documento web, y salta de forma segura a la siguiente línea para continuar analizando el resto del archivo.

6. Traducción a HTML

Para cumplir con el requerimiento de transformar los scripts `.smart` a documentos HTML, hemos implementado una estrategia de Traducción Dirigida por la Sintaxis utilizando las acciones semánticas de Bison en el archivo `Parser.y`.

Esta técnica nos permite integrar bloques de código C directamente en las reglas de producción de nuestra gramática. De esta manera, cada vez que el analizador sintáctico reduce una regla válida, se ejecutan las acciones asociadas que escriben el código HTML correspondiente en un archivo de salida.

6.1 Implementación

La traducción se lleva a cabo mediante 2 módulos importantes:

Gestión de Archivos (`main.c`): El programa principal es el encargado de inicializar el flujo de escritura del archivo HTML, asignándole el mismo nombre que el archivo fuente (con extensión `.html` o `.htm`). Además, implementamos una lógica de seguridad que, en caso de detectar errores sintácticos durante el proceso, destruye el archivo generado mediante la función `remove()` para evitar la entrega de un código incompleto o corrupto.

Acciones Semánticas (`Parser.y`):

- **Sensores:** Se generan bloques `<div>` con un borde verde de 1px y padding de 20px. El nombre del sensor y sus valores se inyectan en etiquetas específicas para mantener una estructura semántica.
- **Actuadores:** Se utilizan bloques `<div>` con borde gris y padding de 20px, estructurando sus atributos internos mediante listas HTML `<ul>` y elementos de lista `<li>`, garantizando una presentación clara y ordenada.

- **Correos Electrónicos:** Para los atributos de tipo email, se transforman automáticamente en enlaces hipervinculados utilizando la etiqueta <a> con el protocolo mailto:, permitiendo un contacto directo a través del navegador.

Esta arquitectura es altamente eficiente y escalable. Al inyectar el código HTML durante la reducción sintáctica, minimizamos el uso de memoria adicional, ya que no requerimos mantener un Árbol Sintáctico Abstracto (AST) completo en memoria para realizar la traducción. Además, la separación de estilos mediante las configuraciones CSS inyectadas asegura que la salida sea visualmente coherente con los requisitos de la cátedra.

7. Funciones Auxiliares

7.1. Módulo Principal main.c

int es_archivo_valido(const char *nombre):

- **Propósito:** Valida que el archivo fuente proporcionado por el usuario contenga la extensión requerida (.smart).
- **Implementación:** Utiliza strchr para buscar el último punto en la cadena del nombre de archivo y strcmp para realizar la comparación. Es la primera barrera de control de entrada del sistema.

int main(int argc, char *argv[]):

- **Propósito:** Actúa como la mente principal del intérprete.
- **Funcionalidades auxiliares:**
 - Gestiona la apertura y cierre del puntero global f_html.
 - Inicializa la estructura base del archivo HTML (encabezados, estilos CSS inyectados mediante fprintf).
 - Invoca la función yyparse() para ejecutar el proceso de análisis.
 - Implementa la lógica de limpieza final: decide si conservar o eliminar el archivo .html generado basándose en la variable global cant_errores.

7.2. Analizador lexico Lexer.l

YY_USER_ACTION:

- **Propósito:** Mecanismo auxiliar que se ejecuta automáticamente tras el reconocimiento de cada token.
- **Funcionalidad:** Realiza la concatenación del lexema actual (yytext) en el buffer global linea_actual. Esto permite mantener un registro del contexto completo de la línea que se está analizando, esencial para reportar errores de forma precisa.

Configuración de Entorno, Directivas de preprocesamiento:

- Incluye bloques condicionales (**#ifdef _WIN32**) que invocan funciones de la API de Windows (**SetConsoleOutputCP(65001)**). Su función auxiliar es configurar la consola para interpretar correctamente la codificación UTF-8, garantizando que símbolos como el grado (°) de temperatura sean procesados correctamente.

7.3. Analizador Sintáctico Parser .y

void yyerror(const char *s):

- **Propósito:** Función de manejo de errores sintácticos requerida por Bison.
- **Funcionalidad:** Ante una inconsistencia sintáctica, esta función incrementa el contador global `cant_errores` e imprime un mensaje en consola notificando que se ha detectado un error durante el proceso de derivación.

8. Modo de Obtención y Estructura de Proyecto

Para la construcción del intérprete, se ha utilizado una arquitectura basada en herramientas de generación de compiladores (Flex y Bison), lo que permite separar la lógica de análisis léxico de la estructura sintáctica.

8.1. Proceso de Construcción

El intérprete se obtiene mediante un proceso de compilación de tres etapas. Este flujo asegura que, ante cualquier modificación en la gramática o en los patrones léxicos, el sistema pueda regenerarse automáticamente:

1. **Generación del Analizador Léxico:** Se procesa **Lexer.l** con flex para generar el archivo fuente en C **lex.yy.c**.
2. **Generación del Analizador Sintáctico:** Se procesa **Parser.y** con bison (usando el flag `-d`) para generar **Parser.tab.c** y **Parser.tab.h** (definiciones de tokens compartidas).
3. **Compilación y Enlazado:** Se compilan todos los archivos fuente (**main.c**, **lex.yy.c**, **Parser.tab.c**) utilizando un compilador de C (gcc) para obtener el ejecutable final.

Proceso de Compilación:

1. Generar el código en C del lexer: **flex Lexer.l**
2. Generar el código en C del parser: **bison -d Parser.y**
3. Compilar el proyecto completo:
 - **En Linux:** `gcc -o Interprete_linux Parser.tab.c lex.yy.c main.c`
 - **En Windows (usando MinGW):** `gcc -o Interprete_win.exe Parser.tab.c lex.yy.c main.c`

8.2 Explicación de Archivos fuentes

El proyecto está compuesto por los siguientes archivos:

Archivo	Rol	Descripción
main.c	Driver	Es el punto de entrada del programa. Gestiona la validación del archivo de entrada (.smart), inicializa el puntero global <code>f_html</code> y orquesta la ejecución del <code>yyparse()</code> .

Lexer.l	Especificación Léxica	Contiene las reglas para identificar cada token del lenguaje (palabras reservadas, sensores, actuadores, unidades). Define la estructura del autómata léxico.
Parser.y	Definición Gramatical	Contiene la Gramática Libre de Contexto (GLC). Incluye las reglas de producción y las acciones semánticas en C que ejecutan la traducción a HTML.
Parser.tab.h	Cabecera de Tokens	Archivo generado automáticamente. Contiene las definiciones numéricas de los tokens, permitiendo que tanto el Lexer como el Parser compartan un lenguaje común.
lex.yy.c	Fuente del Lexer	Archivo generado automáticamente. Código fuente C generado por Flex a partir de Lexer.l.

9. Modos de Ejecución del Intérprete

9.1 Modo Interactivo

Este modo es obligatorio y fundamental para la etapa del Analizador Léxico. Permite procesar la entrada de forma directa desde la consola.

Rol en el Lexer: Es la herramienta de validación técnica por excelencia. Permite ingresar lexemas individuales o sentencias cortas para verificar que el autómata manual los clasifique correctamente en tiempo real. Es indispensable durante el desarrollo para asegurar que no existan errores en las transiciones de estado.

Rol en el Parser: Se mantiene disponible como funcionalidad de soporte. Es ideal para probar rápidamente reglas gramaticales específicas o fragmentos sintácticos aislados sin la necesidad de escribir y gestionar archivos físicos.

Ejemplos de ejecución: (estando en la carpeta principal)

Comando de ejecución en Windows (CMD / PowerShell):

- `.\bin\lexer_win.exe`

Comando de ejecución en Linux:

- `./bin/lexer_linux`

9.2 Modo Lectura de Archivo

Este modo es el estándar operativo y es el más importante para la etapa del Analizador Sintáctico y la Traducción a HTML.

Rol en el Parser/Traductor: Es el modo de producción. Dado que el objetivo final es la generación de un archivo .html a partir de un script .smart completo, este modo permite procesar grandes volúmenes de reglas lógicas. Aquí es donde la gramática se despliega en su totalidad y el traductor genera la estructura HTML completa.

Rol en el Lexer: Está presente para realizar pruebas de carga y verificar el comportamiento del lexer frente a archivos de prueba largos, asegurando que el reporte de errores y el conteo de líneas sean consistentes en archivos reales.

Ejemplos de ejecución: (estando en la carpeta principal)

Comando de ejecución en Windows (CMD / PowerShell):

- `.\bin\Interprete_win.exe .\prueba\ejemplo.smart`

Comando de ejecución en Linux:

- `./bin/Interprete_linux ./prueba/ejemplo.smart`

10. Conclusiones

Completar después

11. Bibliografía

Para la realización de este Trabajo Práctico Integrador, se han consultado diversas fuentes documentales, recursos audiovisuales y herramientas de asistencia técnica que permitieron la comprensión de los conceptos teóricos y la implementación del intérprete.

11.1. Documentación Obligatoria de la Cátedra

Documentación técnica provista por el equipo docente para el desarrollo del proyecto:

- UTN FRRE. (2026). SSL2026-TPI_InterpreteSmartHome.pdf: *Especificaciones técnicas y requerimientos del TPI*. Cátedra de Sintaxis y Semántica de los Lenguajes.

11.2. Software

Recursos utilizados para la descarga de herramientas:

- MinGW / GCC (2026). Compilador GCC para Windows. Disponible en: [MinGW - Minimalist GNU for Windows download | SourceForge.net](https://sourceforge.net/projects/mingw/)

11.3. Recursos Audiovisuales (Tutoriales)

Videos consultados para la instalación, configuración y resolución de errores comunes:

- [Santiago Andrade](https://www.youtube.com/watch?v=yF04oPCpy7c) (2019). FLEX, BISON Y MinGW: DESCARGA, INSTALACIÓN Y EJEMPLO DE COMPILACIÓN. <https://www.youtube.com/watch?v=yF04oPCpy7c>
- Charly Cimino (2022). Instalación de MinGW para compilar en C/C++ . https://www.youtube.com/watch?v=GEZLjRSY_m8&t=290s
- Dami AI Studio (2021). FLEX Analizador Lexico - Explicacion, Compilacion y Pruebas. <https://www.youtube.com/watch?v=AyB7gVNor9U&t=283s>

- Benjamin Escobar (2025). Analizador Léxico con Flex.
<https://www.youtube.com/watch?v=114Pt0mG2nc&t=467s>
- [Universitat Politècnica de València - UPV](#) (2011). Bison. Desarrollo de un analizador sintáctico usando Bison | 5/13 | UPV. <https://www.youtube.com/watch?v=vfDLKxbTGQo>

11.4. Asistencia por Inteligencia Artificial (IA)

Se declara el uso de herramientas de IA como colaboradores en el proceso de desarrollo:

- **Google Gemini (2026)**. IA colaboradora. Utilizada como tutor técnico y asistente en la revisión de código, optimización de la gramática (Parser.y), depuración de errores lógicos en el autómata (Lexer.l) y asistencia en el proceso de investigación para la traducción a HTML.
- **Google NotebookLM (2026)**. Herramienta de gestión de contexto. Utilizada para organizar, sintetizar y realizar consultas sobre la documentación oficial de la cátedra, facilitando la extracción de requerimientos específicos del TPI.

12. Anexos